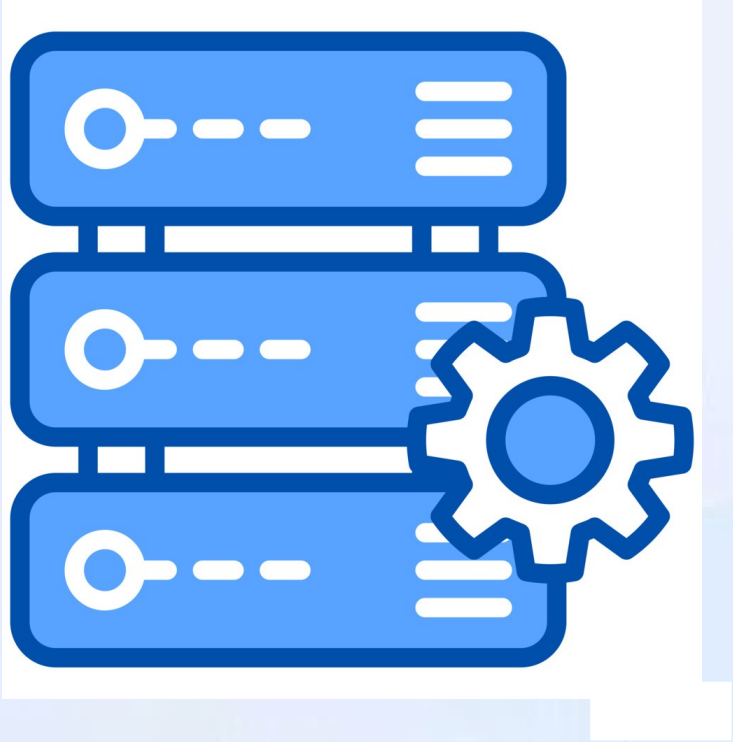
操作系统的进程调度

理解进程调度的基本原理与算法

课程概览



01 进程调度的基本概念

02 常见的调度算法

03 调度算法的比较与选择

什么是进程调度？

定义：进程调度是操作系统的核心功能，负责决定就绪队列中的哪个进程应获得 CPU 的执行权。

目标：提高 CPU 利用率、提升系统吞吐量、缩短进程响应时间、保证系统公平性。

发生时机：进程创建、进程退出、进程阻塞（如 I/O 请求）、进程唤醒、时间片用完。

进程调度的三个层次

高级调度（作业调度）

- 从后备队列选作业调入内存
- 为作业创建进程
- 决定系统的并发度

中级调度（内存调度）

- 挂起 / 唤醒进程（内外存交换）
- 提高内存利用率
- 提升系统吞吐量

低级调度（进程调度）

- 从就绪队列选进程分配 CPU
- 最基本、最频繁的调度
- 毫秒级执行频率

抢占式与非抢占式调度

非抢占式

- 机制：进程运行至完成或阻塞才释放 CPU
- 优点：实现简单，系统开销小
- 缺点：系统响应性差，长进程阻塞短进程
- 场景：批处理系统，不适用于分时系统

抢占式

- 机制： OS 按原则强行剥夺 CPU 分配给其他进程
- 优点：响应快，实时性好，资源利用率高
- 缺点：实现复杂，存在进程切换额外开销
- 场景：分时系统、实时系统

主要调度算法

先来先服务(FCFS)

- 按到达顺序调度
- 优点：公平、简单
- 缺点：长作业有利，短作业等待长

短作业优先(SJF)

- 选运行时间最短优先
- 优点：平均等待时间最短
- 缺点：需预知时间，长作业饥饿

优先级调度

- 按优先级高低调度
- 优点：照顾重要进程
- 缺点：低优先级进程饥饿

时间片轮转(RR)

- 运行一个时间片后抢占
- 优点：公平、响应快
- 缺点：时间片大小影响性能

调度算法性能比较

算法	吞吐量	响应时间	公平性	实现复杂度
FCFS	中等	差	好	简单
SJF	高	中等	差	中等
RR	中等	好	很好	中等
多级反馈队列	高	好	好	复杂

调度算法的选择标准

- **系统类型**：批处理（高吞吐量）、交互式（短响应）、实时（截止时间保证）
- **性能指标**：吞吐量、响应时间、周转时间、公平性等
- **实现复杂度**：算法的实现难度和运行开销
- **特殊需求**：实时性要求、安全性要求等

进阶案例：Linux 内核的进程调度

CFS 完全公平调度器： Linux 2.6.23 起默认采用，摒弃固定时间片，追求“完全公平”的调度理念。

核心思想（vruntime）：基于虚拟运行时间调度，优先级越高权重越大，vruntime 增长越慢，从而获得更多 CPU 时间。

数据结构（红黑树）：用红黑树组织进程，按键值 vruntime 排序，始终选择最左侧节点（vruntime 最小）运行，决策高效。

优势与特性：按权重比例分配资源，避免低优先级任务饥饿，具备优秀的交互性与扩展性，适应多场景负载。

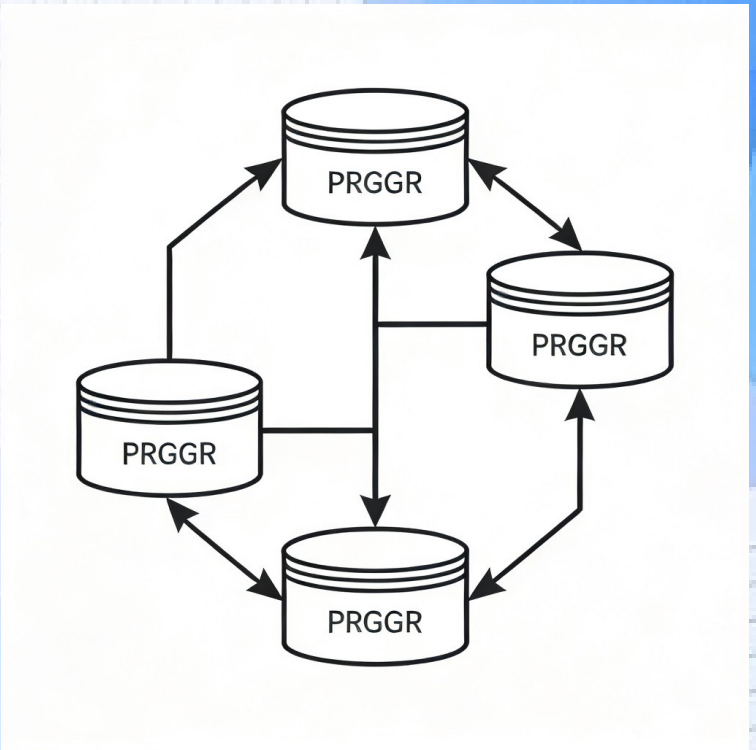
多级反馈队列调度

基本思想：

设置多个就绪队列，各队列优先级和时间片不同。新进程入第一级队列按 RR 调度，未完成则降级。仅高优先级队列为空时调度低优先级进程。

算法优势：

结合了 RR 和优先级调度的优点，能满足各类用户需求，是目前公认的较好调度算法。



实时系统调度

硬实时系统：任务必须在规定的截止时间内完成，否则会导致严重后果（如工业控制、航空航天）。

软实时系统：任务尽量在截止时间内完成，偶尔错过影响不大（如视频播放、在线游戏）。

常用算法：

- 最早截止时间优先（EDF）：选择截止时间最早的任务优先执行。
- 速率单调调度（RM）：根据任务周期（速率）分配优先级，周期越短，优先级越高。

课程总结

进程调度是操作系统的核心功能，

合理选择调度算法能显著提升系统性能。

不同的调度算法适用于不同的应用场景，

需要在系统类型、性能指标和实现复杂度之间做出权衡。